

Università degli Studi di Napoli Federico II

Corso di Ingegneria del Software — Canale 2

BugBoard26

Documento di Specifica dei Requisiti Software (SRS)

Versione: 2.0

Data: 19 febbraio 2026

Stato: Consegna progetto completo

Anno Accademico 2025/2026

Indice

1	Introduzione	4
1.1	Scopo del documento	4
1.2	Descrizione del sistema	4
1.3	Riferimenti	4
2	Glossario	4
3	Individuazione e caratterizzazione degli utenti (attori)	6
4	Modellazione dei Casi d'Uso	6
4.1	Tabella dei Casi d'Uso per Attore	6
5	Tabella dei Casi d'Uso	6
5.1	Descrizione dettagliata dei casi d'uso	8
5.1.1	Utente non registrato	8
5.1.2	Utente registrato (USER)	8
5.1.3	Utente ReadOnly	9
5.1.4	Admin	9
5.1.5	Sistema	10
5.2	Diagrammi UML dei Casi d'Uso	10
5.2.1	Diagramma Generale del Sistema	10
5.2.2	Diagramma Dettagliato — Gestione Bug	10
5.2.3	Diagramma Dettagliato — Amministrazione	11
5.3	Convenzioni grafiche	11
6	Formalizzazione dei Casi d'Uso (Cockburn)	11
6.1	Caso d'Uso 1: Creazione nuovo bug	11
6.2	Caso d'Uso 2: Assegnazione bug a un utente	12
6.3	Caso d'Uso 3: Risoluzione e chiusura bug	13
6.4	Caso d'Uso 4: Generazione report mensile (Admin)	14
7	Requisiti Non Funzionali e di Sistema	15
7.1	Usabilità	16
7.2	Prestazioni	16
7.3	Sicurezza	16
7.4	Affidabilità e Disponibilità	17
7.5	Manutenibilità e Portabilità	17
7.6	Vincoli di Sistema	17
8	Prototipazione visuale attraverso Mock-up	17
8.1	Scelte Progettuali	18
8.1.1	Layout e Navigazione	18
8.1.2	Palette Colori e Badge	18
8.1.3	Componenti UI riutilizzati	18
8.2	Mock-up delle interfacce utente	18
8.2.1	Mock-up 1 — Pagina di Login	19
8.2.2	Mock-up 2 — Pagina di Registrazione	20

8.2.3	Mock-up 3 — Home / Dashboard	20
8.2.4	Mock-up 4 — Lista Bug con Filtri	21
8.2.5	Mock-up 5 — Dettaglio Bug	22
8.2.6	Mock-up 6 — Form Creazione/Modifica Bug [UC01]	23
8.2.7	Mock-up 7 — Assegnazione Bug [UC02]	24
8.2.8	Mock-up 8 — Notifiche	25
8.2.9	Mock-up 9 — Profilo Utente	25
8.2.10	Mock-up 10 — Report Mensile [UC04]	26
8.2.11	Mock-up 11 — Gestione Utenti (Admin)	27
8.2.12	Mock-up 12 — Archivio Bug	27
8.2.13	Mock-up 13 — Export Dati	28
8.3	Osservazioni emerse dalla prototipazione	28
9	Design del Sistema	29
9.1	Descrizione dell'architettura	29
9.1.1	Diagramma architetturale	29
9.2	Criteri di design adottati	29
9.3	Scelte tecnologiche	30
9.3.1	Frontend	30
9.3.2	Backend	31
9.3.3	Testing	31
9.3.4	DevOps e Deployment	32
9.4	Comunicazione tra i livelli	32
9.5	Sicurezza	32
10	Design del Software	33
10.1	Schema per la persistenza dati	33
10.1.1	Tabella users	33
10.1.2	Tabella bugs	33
10.1.3	Tabella comments	33
10.1.4	Tabella history	34
10.1.5	Tabella labels	34
10.1.6	Tabella di join bug_labels	34
10.1.7	Diagramma E-R semplificato	34
10.2	Diagramma delle classi di design	34
10.3	Scelte di software design	35
10.3.1	Pattern Repository	35
10.3.2	Pattern Service Layer	35
10.3.3	Record Java per i DTO	36
10.3.4	Specification Pattern per le query	36
10.3.5	Audit Trail tramite History	36
10.3.6	Autenticazione stateless con JWT	36
10.3.7	Gestione delle autorizzazioni	36
10.4	Evidenza dell'uso di strumenti di versioning	36

11 Test Plan – Gestione Bug	37
11.1 Obiettivo	37
11.2 Funzionalità sotto test	37
11.3 Approccio di testing	37
11.4 Casi di test pianificati	37
11.4.1 create(BugCreateRequest, UUID)	37
11.4.2 assign(UUID, AssignRequest, UUID)	37
11.4.3 patch(UUID, BugPatchRequest, UUID, boolean)	38
11.5 Criteri di ingresso e uscita	38
11.6 Ambiente di esecuzione	38
12 Strategie di Testing	38
12.1 Metodi testati	38
12.2 Classi di equivalenza	38
12.2.1 create(BugCreateRequest, UUID)	39
12.2.2 assign(UUID, AssignRequest, UUID)	39
12.2.3 patch(UUID, BugPatchRequest, UUID, boolean)	39
12.3 Copertura strutturale	39
12.4 Tecnica di isolamento: Mock Objects	40
12.5 Riepilogo dei risultati	40
13 Report di Qualità del Codice (Backend)	40
13.1 Strumenti di analisi utilizzati	40
13.2 Metriche generali	41
13.3 Distribuzione del codice per package	41
13.4 Punti di forza	41
13.5 Code smell e aree di miglioramento identificate	42
13.5.1 Severità alta	42
13.5.2 Severità media	42
13.5.3 Severità bassa	42
13.6 Riepilogo	42

1 Introduzione

1.1 Scopo del documento

Il presente documento costituisce la Specifica dei Requisiti Software (SRS) per il sistema **BugBoard26**, sviluppato nell'ambito del corso di Ingegneria del Software. Il documento descrive le funzionalità del sistema, i suoi attori, i casi d'uso formalizzati, i requisiti non funzionali e fornisce una prima prototipazione visuale dell'interfaccia utente.

1.2 Descrizione del sistema

BugBoard26 è un'applicazione web per il **tracciamento e la gestione collaborativa di segnalazioni di bug**. Il sistema consente a team di sviluppo di:

- segnalare bug, feature request e miglioramenti;
- assegnare le segnalazioni agli sviluppatori;
- tracciare il ciclo di vita di ogni segnalazione (apertura, presa in carico, risoluzione, archiviazione);
- aggiungere commenti e allegati fotografici;
- visualizzare statistiche e generare report mensili sull'attività del team.

1.3 Riferimenti

- A. Cockburn, *Writing Effective Use Cases*, Addison-Wesley, 2001
- Traccia progetto corso Ingegneria del Software — Canale 2
- Documentazione di riferimento: DietiDeals24 (esempio fornito dal docente)

2 Glossario

Di seguito sono riportati i termini tecnici e di dominio utilizzati nel presente documento e nel sistema BugBoard26.

Admin	Utente con privilegi amministrativi completi. Può gestire utenti, accedere a tutti i bug del sistema, generare report e modificare configurazioni.
Allegato	File immagine (JPEG, PNG, GIF, WebP, dimensione massima 5 MB) caricato in associazione a un bug per documentarne visivamente il problema o la soluzione.
Archiviazione	Operazione che sposta un bug in stato <i>Archived</i> , rimuovendolo dalla lista attiva senza eliminarlo definitivamente dal sistema.
Assegnatario	Utente registrato responsabile della risoluzione di un determinato bug.

Bug	Segnalazione di un comportamento anomalo, errato o inatteso di un sistema software. Nel contesto di BugBoard26 il termine indica genericamente qualsiasi segnalazione inserita nel sistema, indipendentemente dal tipo (Bug, Feature, Enhancement).
BugBoard26	Il sistema software oggetto di questo documento. Applicazione web per il tracciamento e la gestione collaborativa delle segnalazioni di bug.
Caso d'uso	Descrizione di un'interazione tra un attore e il sistema per raggiungere un obiettivo specifico. Formalizzato secondo il metodo di Alistair Cockburn.
Commento	Testo inserito da un utente in associazione a un bug per fornire aggiornamenti, chiarimenti o informazioni aggiuntive.
Docker	Piattaforma di containerizzazione utilizzata per impacchettare e distribuire i componenti dell'applicazione (frontend e backend) in modo indipendente dall'ambiente host.
Enhancement	Tipo di segnalazione che indica una richiesta di miglioramento di una funzionalità già esistente. Distinto da un Bug (errore) e da una Feature (nuova funzionalità).
Feature	Tipo di segnalazione che indica la richiesta di implementazione di una nuova funzionalità nel sistema.
History (Storico)	Registro cronologico di tutte le modifiche apportate a un bug nel tempo: cambi di stato, assegnazioni, commenti, aggiunta di allegati.
JWT (JSON Web Token)	Standard aperto (RFC 7519) utilizzato per autenticare in modo sicuro le richieste HTTP tra frontend e backend. Il token viene generato al login e incluso in ogni richiesta successiva.
Label	Etichetta testuale opzionale associabile a un bug per categorizzarlo ulteriormente (es. "frontend", "performance", "regression").
Priorità	Livello di urgenza di un bug. I valori previsti sono: <i>Low</i> , <i>Medium</i> , <i>High</i> , <i>Critical</i> .
ReadOnly (Utente)	Ruolo con accesso in sola lettura. L'utente ReadOnly può visualizzare bug, commenti e allegati, ma non può creare, modificare o commentare.
Report Mensile	Documento generato dal sistema contenente statistiche aggregate sull'attività del periodo selezionato: numero di bug creati, risolti, tempo medio di risoluzione, distribuzione per priorità e tipo.
Ruolo	Insieme di permessi associato a un utente. I ruoli previsti sono: <i>USER</i> (utente standard), <i>READONLY</i> (sola lettura), <i>ADMIN</i> (amministratore).

Stato (Status)

Fase del ciclo di vita di un bug. Gli stati previsti sono:

- *Todo*: bug appena creato, non ancora preso in carico
- *In Progress*: bug in fase di risoluzione
- *Resolved*: bug risolto
- *Archived*: bug archiviato

Tipo (Type) Categoria della segnalazione. I valori previsti sono: *Bug*, *Feature*, *Enhancement*.

USER (Utente registrato)

Ruolo standard. L'utente può creare bug, commentare, caricare allegati, assegnare bug (se Admin) e gestire il proprio profilo.

3 Individuazione e caratterizzazione degli utenti (attori)

Il sistema prevede quattro tipologie di attori umani e un attore di sistema:

Utente non registrato (Guest)

Chiunque acceda all'applicazione senza credenziali. Può accedere alla pagina di login e di registrazione. Non ha accesso ad alcuna funzionalità interna.

Utente registrato (USER)

Utente autenticato con ruolo standard. Può creare e modificare bug, aggiungere commenti e allegati, visualizzare lo storico, gestire il proprio profilo.

Utente ReadOnly

Utente autenticato con accesso in sola lettura. Può visualizzare bug, commenti e allegati, ma non può eseguire operazioni di scrittura.

Admin

Utente con privilegi amministrativi completi. Eredita tutte le funzionalità di USER e può in aggiunta: gestire gli utenti del sistema, assegnare bug, visualizzare metriche, generare report, esportare dati.

Sistema

Attore non umano che esegue operazioni automatiche: invio di notifiche, generazione dello storico modifiche, validazione degli allegati.

4 Modellazione dei Casi d'Uso

4.1 Tabella dei Casi d'Uso per Attore

5 Tabella dei Casi d'Uso

Attore	Casi d'uso
Utente non registrato	• Registrazione al sistema tramite email • Visualizzazione informazioni pubbliche dell'applicazione • Accesso alla pagina di login
Utente registrato (USER)	• Login/logout al sistema • Visualizzazione lista bug con filtri (tipo, stato, priorità, label) • Visualizzazione dettagli singolo bug • Creazione nuovo bug • Modifica bug propri o assegnati • Aggiunta commenti ai bug • Upload allegati immagini ai bug • Assegnazione bug ad altri utenti • Archiviazione bug risolti • Marcatura bug come duplicati • Visualizzazione storico modifiche bug • Gestione profilo personale • Cambio password
Utente ReadOnly	• Login al sistema • Visualizzazione lista bug con filtri • Visualizzazione dettagli singolo bug • Visualizzazione commenti e allegati • Visualizzazione storico modifiche • Visualizzazione profilo proprio (sola lettura)
Admin	• Tutti i casi d'uso dell'Utente registrato • Gestione completa utenti (visualizzazione, modifica ruoli, disabilitazione) • Visualizzazione metriche di sistema • Generazione report mensili • Esportazione dati in formato Excel • Accesso completo a tutti i bug del sistema • Gestione label e categorie • Configurazione parametri di sistema
Sistema	• Invio notifiche automatiche su cambiamenti bug • Archiviazione automatica bug dopo periodo di inattività • Generazione automatica storico modifiche • Validazione automatica allegati (tipo e dimensione) • Pulizia automatica dati obsoleti • Backup automatico dati

Attore	Casi d'uso
--------	------------

Tabella 1: Tabella completa dei casi d'uso per BugBoard26

5.1 Descrizione dettagliata dei casi d'uso

5.1.1 Utente non registrato

- **Registrazione al sistema tramite email:** L'utente può creare un nuovo account fornendo email, password e informazioni personali. Il sistema invia una email di conferma per verificare l'indirizzo.
- **Visualizzazione informazioni pubbliche:** Accesso alla pagina principale dell'applicazione con informazioni generali sul servizio di bug tracking.
- **Accesso alla pagina di login:** Possibilità di accedere al sistema utilizzando credenziali esistenti.

5.1.2 Utente registrato (USER)

- **Login/logout al sistema:** Autenticazione tramite credenziali email/password con generazione di token JWT per sessioni sicure.
- **Visualizzazione lista bug con filtri:** Consultazione dell'elenco completo dei bug con possibilità di filtrare per tipo, stato, priorità, label e ricerca testuale.
- **Visualizzazione dettagli singolo bug:** Accesso completo alle informazioni di un bug specifico, inclusi descrizione, commenti, allegati e storico modifiche.
- **Creazione nuovo bug:** Inserimento di una nuova segnalazione con descrizione dettagliata, assegnazione priorità, tipo e label appropriate.
- **Modifica bug propri o assegnati:** Possibilità di aggiornare informazioni, stato e dettagli dei bug creati dall'utente o assegnati al suo account.
- **Aggiunta commenti ai bug:** Inserimento di commenti testuali per fornire aggiornamenti, soluzioni o richiedere chiarimenti.
- **Upload allegati immagini ai bug:** Caricamento di immagini (JPEG, PNG, GIF, WebP) fino a 5MB per documentare visivamente il problema.
- **Assegnazione bug ad altri utenti:** Trasferimento della responsabilità di un bug a un altro utente registrato nel sistema.
- **Archiviazione bug risolti:** Spostamento dei bug risolti in stato archiviato per mantenere pulita la lista attiva.
- **Marcatura bug come duplicati:** Collegamento di un bug a un altro esistente quando viene identificato come duplicato.
- **Visualizzazione storico modifiche bug:** Consultazione cronologica di tutte le modifiche apportate a un bug nel tempo.

- **Gestione profilo personale:** Modifica delle informazioni personali, preferenze e impostazioni dell'account.
- **Cambio password:** Aggiornamento della password di accesso con validazione di sicurezza.

5.1.3 Utente ReadOnly

- **Login al sistema:** Accesso con credenziali per utenti con privilegi di sola lettura.
- **Visualizzazione lista bug con filtri:** Consultazione dell'elenco bug con possibilità di applicare filtri di ricerca.
- **Visualizzazione dettagli singolo bug:** Accesso in sola lettura alle informazioni complete di un bug.
- **Visualizzazione commenti e allegati:** Lettura di tutti i commenti e download/visualizzazione degli allegati associati.
- **Visualizzazione storico modifiche:** Consultazione della cronologia delle modifiche senza possibilità di intervento.
- **Visualizzazione profilo proprio:** Accesso limitato alle informazioni del proprio profilo utente.

5.1.4 Admin

- **Tutti i casi d'uso dell'Utente registrato:** L'amministratore eredita tutte le funzionalità dell'utente standard.
- **Gestione completa utenti:** Creazione, modifica, disabilitazione e gestione ruoli degli utenti del sistema.
- **Visualizzazione metriche di sistema:** Accesso a dashboard con statistiche, grafici e KPI del sistema.
- **Generazione report mensili:** Creazione di report dettagliati sull'attività del sistema per periodo specificato.
- **Esportazione dati in formato Excel:** Estrazione completa dei dati in formato spreadsheet per analisi esterne.
- **Accesso completo a tutti i bug:** Possibilità di visualizzare e modificare qualsiasi bug del sistema indipendentemente dall'assegnazione.
- **Gestione label e categorie:** Creazione, modifica ed eliminazione delle etichette e categorie di classificazione.
- **Configurazione parametri di sistema:** Modifica delle impostazioni globali, limiti e comportamenti dell'applicazione.

5.1.5 Sistema

- **Invio notifiche automatiche:** Meccanismo automatico di notifica via email o push per aggiornamenti sui bug seguiti.
- **Archiviazione automatica:** Processo schedato per archiviare automaticamente bug inattivi dopo un periodo configurabile.
- **Generazione automatica storico:** Registrazione automatica di ogni modifica apportata ai bug nel database.
- **Validazione automatica allegati:** Controllo automatico di tipo, dimensione e sicurezza dei file caricati.
- **Pulizia automatica dati obsoleti:** Eliminazione periodica di dati temporanei, cache e informazioni non più necessarie.
- **Backup automatico dati:** Salvataggio automatico periodico del database e dei file allegati per disaster recovery.

5.2 Diagrammi UML dei Casi d'Uso

5.2.1 Diagramma Generale del Sistema

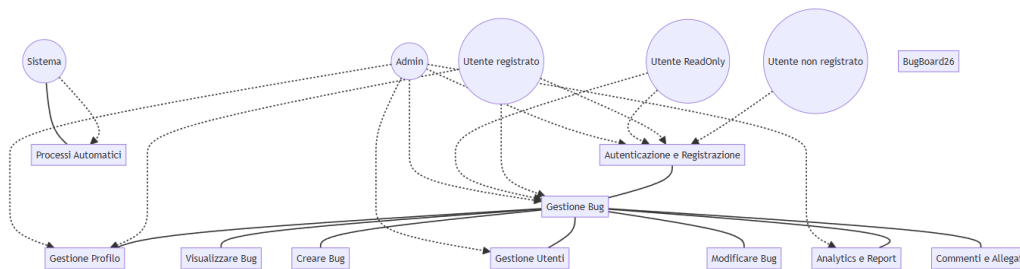


Figura 1: Diagramma generale dei casi d'uso — BugBoard26

5.2.2 Diagramma Dettagliato — Gestione Bug

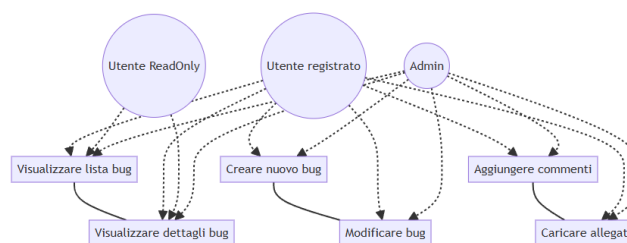


Figura 2: Diagramma dettagliato — Gestione Bug

5.2.3 Diagramma Dettagliato — Amministrazione

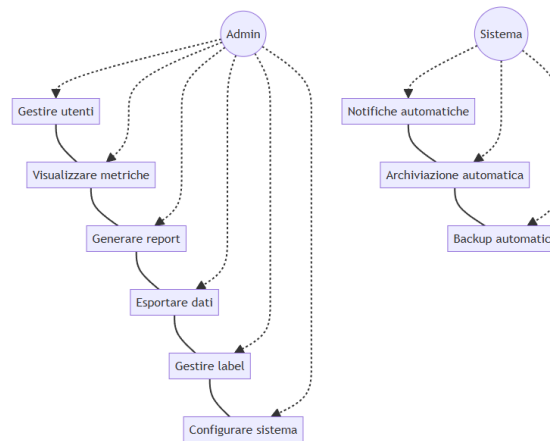


Figura 3: Diagramma dettagliato — Amministrazione e funzioni di sistema

5.3 Convenzioni grafiche

- **Attori** (omino stilizzato o rettangolo): entità esterne che interagiscono con il sistema
- **Casi d'uso** (ellissi): funzionalità offerte dal sistema
- **Linee solide**: relazione di associazione tra attore e caso d'uso
- **Frecce tratteggiate** **«include»**: il caso d'uso base include sempre il caso d'uso puntato
- **Frecce tratteggiate** **«extend»**: il caso d'uso puntato estende opzionalmente il caso d'uso base

6 Formalizzazione dei Casi d'Uso (Cockburn)

In questa sezione vengono formalizzati quattro casi d'uso significativi, secondo il formalismo tabellare di Alistair Cockburn. Sono stati esclusi i casi d'uso relativi a registrazione e autenticazione, come indicato nelle istruzioni della traccia.

6.1 Caso d'Uso 1: Creazione nuovo bug

Campo	Descrizione
USE CASE #01	CREAZIONE NUOVO BUG
Goal	L'utente vuole segnalare un nuovo bug nel sistema
Preconditions	L'utente deve essere autenticato (ruolo USER o ADMIN)
Success End Condition	Il bug è creato con successo e registrato con un ID univoco

Campo	Descrizione	
Failed End Condition	Il bug non viene creato a causa di errori di validazione o problemi di sistema	
Primary Actor	Utente registrato (USER)	
DESCRIPTION	Utente	Sistema
Step 1	Accede alla sezione “Nuovo Bug”	Mostra il form con i campi obbligatori
Step 2	Inserisce il titolo del bug	Valida che il titolo non sia vuoto
Step 3	Seleziona il tipo (Bug, Feature, Enhancement)	Carica le categorie disponibili
Step 4	Inserisce la descrizione dettagliata	Fornisce un’area di testo
Step 5	Seleziona la priorità (Low, Medium, High, Critical)	Mostra le opzioni con colori identificativi
Step 6	Aggiunge label/tag opzionali	Permette selezione multipla da lista
Step 7	Carica immagini allegati (opzionale)	Valida formato (JPEG, PNG, GIF, WebP) e dimensione (max 5 MB)
Step 8	Seleziona assegnatario (opzionale)	Mostra lista utenti attivi
Step 9	Clicca su “Crea Bug”	Valida tutti i campi obbligatori
Step 10		Salva il bug nel database
Step 11		Invia notifica all’utente assegnato (se presente)
Step 12		Mostra conferma con l’ID del bug creato
EXTENSIONS	Step	Descrizione
Caricamento immagini fallito	7.a	Mostra errore “Formato non supportato”
	7.b	L’utente può riprovare con file diverso
Validazione campi fallita	9.a	Evidenzia i campi con errori
	9.b	Mostra messaggi di errore per ciascun campo
Utente assegnatario non trovato	8.a	Mostra errore “Utente non trovato”
	8.b	L’utente può eseguire una nuova ricerca

6.2 Caso d’Uso 2: Assegnazione bug a un utente

Campo	Descrizione
USE CASE #02	ASSEGNAZIONE BUG A UN UTENTE
Goal	L’amministratore vuole assegnare o riassegnare un bug a un utente del sistema

Campo	Descrizione	
Preconditions	L'utente deve essere autenticato con ruolo ADMIN; il bug deve esistere nel sistema	
Success End Condition	Il bug risulta assegnato al nuovo utente e la modifica è registrata nello storico	
Failed End Condition	L'assegnazione fallisce per utente di destinazione non valido	
Primary Actor	Admin	
DESCRIPTION	Utente (Admin)	Sistema
Step 1	Visualizza i dettagli di un bug	Mostra la pagina di dettaglio con tutte le informazioni
Step 2	Clicca su "Assegna"	Apri il form di assegnazione
Step 3	Inserisce email o nome del nuovo assegnatario	Fornisce autocomplete con ricerca utenti
Step 4	Seleziona l'utente dalla lista	Valida che l'utente esista e sia attivo
Step 5	Aggiunge una nota (opzionale)	Fornisce un campo testo per la motivazione
Step 6	Clicca su "Conferma Assegnazione"	Verifica i permessi dell'utente richiedente
Step 7		Aggiorna il campo "assegnato a" nel database
Step 8		Registra la modifica nella tabella history
Step 9		Invia notifica al nuovo assegnatario
Step 10		Invia notifica all'assegnatario precedente (se diverso)
Step 11		Mostra messaggio di conferma
EXTENSIONS	Step	Descrizione
Utente non trovato	4.a	Mostra errore "Utente non esistente"
	4.b	L'admin può eseguire una nuova ricerca
Bug già assegnato	6.a	Chiede conferma per il cambio di assegnazione
	6.b	Se confermato, procede con la riassegnazione

6.3 Caso d'Uso 3: Risoluzione e chiusura bug

Campo	Descrizione
USE CASE #03	RISOLUZIONE E CHIUSURA BUG
Goal	L'utente vuole marcare un bug come risolto
Preconditions	L'utente deve essere autenticato e assegnatario del bug; il bug deve essere in stato <i>In Progress</i>

Campo	Descrizione	
Success End Condition	Il bug viene aggiornato allo stato <i>Resolved</i> con data di risoluzione registrata	
Failed End Condition	La risoluzione fallisce per permessi insufficienti o stato del bug non valido	
Primary Actor	Utente registrato (USER) assegnato al bug	
DESCRIPTION	Utente	Sistema
Step 1	Visualizza i dettagli del bug assegnato	Mostra la pagina di dettaglio con lo stato attuale
Step 2	Clicca su “Risolvi Bug”	Verifica che il bug sia in stato <i>In Progress</i>
Step 3		Mostra il form di risoluzione
Step 4	Seleziona il tipo di risoluzione	Mostra opzioni: <i>Fixed</i> , <i>Won't Fix</i> , <i>Duplicate</i> , <i>Cannot Reproduce</i>
Step 5	Inserisce la descrizione della soluzione adottata	Fornisce un'area di testo
Step 6	Carica immagini di conferma (opzionali)	Valida formato e dimensione degli allegati
Step 7	Clicca su “Conferma Risoluzione”	Valida tutti i campi obbligatori
Step 8		Aggiorna lo stato a <i>Resolved</i>
Step 9		Registra la data di risoluzione
Step 10		Crea un record nella tabella history
Step 11		Invia notifiche al creatore e ai follower del bug
Step 12		Mostra la pagina aggiornata con il nuovo stato
EXTENSIONS	Step	Descrizione
Bug non in stato corretto	2.a	Mostra “Bug non può essere risolto in questo stato”
	2.b	Suggerisce azioni alternative disponibili
Tipo risoluzione “Duplicate”	4.a	Richiede la selezione del bug padre
	4.b	Crea automaticamente la relazione di duplicazione
Caricamento immagini fallito	6.a	Mostra errore di formato o dimensione
	6.b	L'utente può proseguire senza allegati

6.4 Caso d'Uso 4: Generazione report mensile (Admin)

Campo	Descrizione	
USE CASE #04	GENERAZIONE REPORT MENSILE	
Goal	L'amministratore vuole generare un report sull'attività del sistema per un mese specifico	
Preconditions	L'utente deve essere autenticato con ruolo ADMIN	
Success End Condition	Il report viene generato e reso disponibile per il download	
Failed End Condition	La generazione fallisce per assenza di dati nel periodo selezionato o errori di sistema	
Primary Actor	Admin	
DESCRIPTION	Utente (Admin)	Sistema
Step 1	Accede alla sezione "Analytics"	Mostra la dashboard con le opzioni di reportistica
Step 2	Seleziona "Report Mensili"	Carica la lista dei mesi disponibili
Step 3	Seleziona mese e anno desiderati	Valida che il periodo esista e contenga dati
Step 4	Clicca su "Genera Report"	Avvia il processo di generazione
Step 5		Raccoglie dati dalle tabelle bugs, users, history
Step 6		Calcola metriche: bug creati, risolti, tempo medio
Step 7		Genera grafici e statistiche aggregate
Step 8		Crea il documento in formato Excel
Step 9		Fornisce il link per il download
Step 10	Scarica il report	Serve il file con nome che include il timestamp
EXTENSIONS	Step	Descrizione
Mese senza dati	3.a	Avvisa "Nessun dato disponibile per il periodo selezionato"
	3.b	Suggerisce periodi alternativi
Errore generazione	8.a	Registra l'errore nel log
	8.b	Mostra un messaggio di errore all'amministratore
	8.c	Suggerisce di riprovare o contattare il supporto

7 Requisiti Non Funzionali e di Sistema

I requisiti non funzionali definiscono le caratteristiche qualitative che il sistema BugBoard26 deve possedere, indipendentemente dalle funzionalità specifiche implementate.

7.1 Usabilità

- **RNF-U1 — Interfaccia responsiva:** L'interfaccia deve essere fruibile su dispositivi desktop, tablet e mobile senza perdita di funzionalità. Il layout si adatta automaticamente alla larghezza dello schermo.
- **RNF-U2 — Feedback immediato:** Ogni operazione che richiede elaborazione (es. creazione bug, caricamento allegati) deve fornire un feedback visivo all'utente entro 500 ms (es. spinner, messaggio di conferma o di errore).
- **RNF-U3 — Messaggi di errore chiari:** I messaggi di errore devono indicare la causa del problema e suggerire un'azione correttiva, senza esporre dettagli tecnici interni.
- **RNF-U4 — Accessibilità:** L'interfaccia deve rispettare le linee guida WCAG 2.1 livello AA, garantendo contrasti adeguati e compatibilità con tecnologie assistive.

7.2 Prestazioni

- **RNF-P1 — Tempo di risposta API:** Le richieste API standard (lista bug, dettaglio bug, login) devono avere un tempo di risposta inferiore a 500 ms in condizioni normali di carico.
- **RNF-P2 — Paginazione:** La lista dei bug è paginata (default 20 elementi per pagina) per evitare il caricamento di dataset molto grandi.
- **RNF-P3 — Dimensione allegati:** I file immagine caricati non possono superare 5 MB ciascuno. Formati accettati: JPEG, PNG, GIF, WebP.
- **RNF-P4 — Generazione report:** La generazione di un report mensile deve completarsi entro 10 secondi per dataset fino a 10 000 bug.

7.3 Sicurezza

- **RNF-S1 — Autenticazione:** Tutti gli endpoint (eccetto login e registrazione) richiedono un token JWT valido. Il token ha una durata configurabile (default: 8 ore).
- **RNF-S2 — Autorizzazione per ruolo:** Le operazioni privilegiate (gestione utenti, report, assegnazione bug) sono accessibili esclusivamente agli utenti con ruolo ADMIN.
- **RNF-S3 — Validazione input:** Tutti i dati in input (lato client e lato server) sono validati prima dell'elaborazione. I file caricati sono verificati per tipo MIME e dimensione.
- **RNF-S4 — Trasmissione sicura:** In produzione, tutte le comunicazioni tra client e server avvengono tramite HTTPS. I cookie di sessione sono configurati con i flag Secure e SameSite.
- **RNF-S5 — Password:** Le password degli utenti sono memorizzate nel database in forma hash (bcrypt). Non vengono mai restituite nelle risposte API.

7.4 Affidabilità e Disponibilità

- **RNF-A1 — Gestione degli errori:** Il backend restituisce codici HTTP appropriati (400, 401, 403, 404, 500) con messaggi descrittivi. Il frontend mostra notifiche comprensibili per l'utente.
- **RNF-A2 — Persistenza dei dati:** I dati sono persistiti su database relazionale (H2 in sviluppo, sostituibile con PostgreSQL in produzione). Nessuna operazione di scrittura avviene esclusivamente in memoria volatile.
- **RNF-A3 — Storico immutabile:** La tabella *History* non permette cancellazione o modifica dei record; ogni cambiamento genera un nuovo record.

7.5 Manutenibilità e Portabilità

- **RNF-M1 — Containerizzazione:** L'applicazione è distribuita tramite Docker (Dockerfile per frontend e backend; `docker-compose.yml` per l'avvio integrato).
- **RNF-M2 — Separazione frontend/backend:** Frontend e backend sono componenti indipendenti che comunicano esclusivamente tramite API REST. Questa architettura consente di aggiornare o sostituire ciascun componente separatamente.
- **RNF-M3 — Configurabilità:** I parametri critici (URL del database, segreto JWT, porta del server, origine CORS) sono configurabili tramite variabili d'ambiente senza modificare il codice sorgente.
- **RNF-M4 — Documentazione API:** Gli endpoint REST sono documentati tramite SpringDoc/OpenAPI (Swagger UI), accessibile in fase di sviluppo.

7.6 Vincoli di Sistema

- **RNF-VS1 — Tecnologie frontend:** React 18, TypeScript, Vite, Tailwind CSS, Radix UI, React Query, React Router.
- **RNF-VS2 — Tecnologie backend:** Java 23, Spring Boot 3.3, Spring Security, Spring Data JPA, database H2 (sviluppo).
- **RNF-VS3 — Browser supportati:** Le ultime due versioni stabili di Chrome, Firefox, Edge e Safari.
- **RNF-VS4 — Requisiti server:** JDK 23+, Maven 3.8+, Node.js 18+ per la build del frontend.

8 Prototipazione visuale attraverso Mock-up

La prototipazione visuale di BugBoard26 è stata realizzata direttamente come applicazione React funzionante, con dati simulati (*mock data*). Le schermate qui mostrate sono catture reali del prototipo interattivo, e non semplici bozzetti grafici. Questo approccio ha permesso di validare il flusso utente in modo concreto fin dalla fase di analisi.

8.1 Scelte Progettuali

8.1.1 Layout e Navigazione

- **Header fisso:** barra superiore con logo, link alle sezioni principali (Dashboard, Bug, Report, Admin) e menu utente con avatar, email e pulsante logout
- **Navigazione mobile:** menu a comparsa (hamburger) con le stesse voci, ottimizzato per touch
- **Layout responsive:** design *mobile-first* realizzato con Tailwind CSS; le griglie si adattano automaticamente a qualsiasi larghezza schermo

8.1.2 Palette Colori e Badge

- **Priorità:** *Critical* (rosso), *High* (arancione), *Medium* (giallo), *Low* (verde)
- **Stato:** *Todo* (grigio chiaro), *In Progress* (blu), *Resolved* (verde), *Archived* (grigio scuro)
- **Tipo:** *Bug* (rosso), *Feature* (viola), *Enhancement* (azzurro)
- **Azioni primarie:** blu (#007BFF); superfici neutre: grigi Tailwind

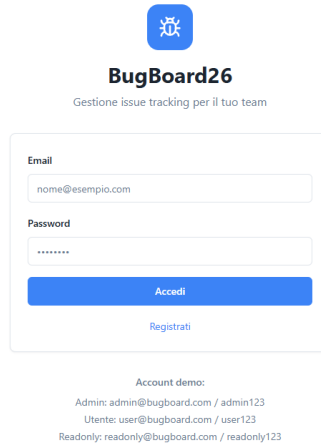
8.1.3 Componenti UI riutilizzati

Tutti i componenti sono basati su **Radix UI** e **shadcn/ui**: Button, Card, Select, Dialog, Badge, Table, Toast, DropdownMenu. Questo garantisce coerenza visiva e accessibilità WCAG 2.1 out-of-the-box.

8.2 Mock-up delle interfacce utente

Di seguito sono illustrate le schermate principali del prototipo, raggruppate per funzionalità. Le schermate contrassegnate con **[UC0X]** corrispondono direttamente ai casi d'uso formalizzati nella sezione precedente.

8.2.1 Mock-up 1 — Pagina di Login



BugBoard26
Gestione issue tracking per il tuo team

Email
nome@esempio.com

Password

[Accedi](#)

[Registrati](#)

Account demo:
Admin: admin@bugboard.com / admin123
Utente: user@bugboard.com / user123
Readonly: readonly@bugboard.com / readonly123

Figura 4: Pagina di Login: form con email e password, pulsante di accesso

- **Scopo:** autenticazione dell'utente al sistema
- **Elementi:** campo email, campo password, pulsante “Accedi”, link alla pagina di registrazione
- **Comportamento:** in caso di credenziali errate viene mostrato un messaggio di errore inline; in caso di successo l'utente viene reindirizzato alla Dashboard

8.2.2 Mock-up 2 — Pagina di Registrazione

← Torna al login



Registrazione

○ La registrazione è gestita dall'amministratore. Questo form è solo dimostrativo.

Nome completo

Mario Rossi

Email

mario@esempio.com

Password

Registrati

Contatta l'amministratore per richiedere un account

Figura 5: Pagina di Registrazione: form con dati personali e creazione account

- **Scopo:** creazione di un nuovo account utente
- **Elementi:** campi nome, email, password, conferma password; pulsante “Registrati”
- **Validazione:** i campi vengono validati in tempo reale con messaggi di errore inline

8.2.3 Mock-up 3 — Home / Dashboard

Dashboard

Benvenuto, Marco Rossi


7
Bug aperti


3
Assegnati a me


2
Risolti


0
Scadenze prossime

Azioni rapide

Visualizza tutti i bug 

Crea nuovo bug 

Gestione utenti Admin

Home Bug Notifiche  Accesso effettuato con successo  Profilo

Figura 6: Home Dashboard: panoramica delle metriche principali e accessi rapidi

- **Scopo:** fornire una panoramica immediata dello stato del sistema dopo il login
- **Elementi:** card metriche (bug aperti, assegnati a me, risolti, scadenze prossime), header con navigazione principale
- **Accessi rapidi:** da qui si può navigare alla lista bug, al form di creazione e ai report

8.2.4 Mock-up 4 — Lista Bug con Filtri

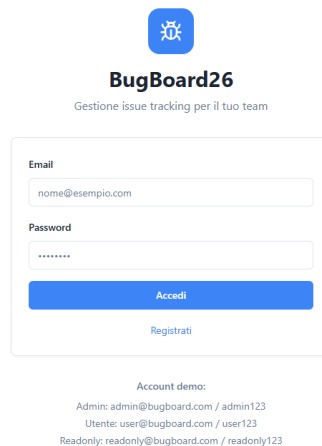


Figura 7: Lista Bug: tabella con filtri per tipo, stato, priorità e ricerca testuale

- **Scopo:** visualizzazione e filtraggio dell'elenco completo dei bug
- **Elementi:** barra filtri orizzontale (tipo, stato, priorità), campo ricerca full-text, tabella bug con badge colorati, paginazione
- **Interazione:** clic su una riga apre il dettaglio del bug; i filtri si applicano in tempo reale

8.2.5 Mock-up 5 — Dettaglio Bug

BugBoard26
Gestione issue tracking per il tuo team

Email
nome@esempio.com

Password

Accedi

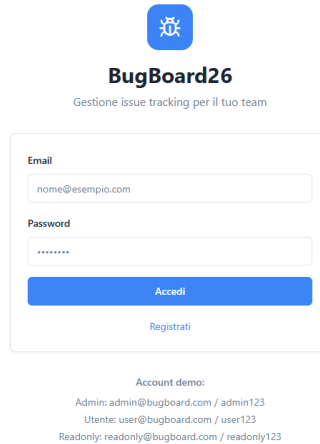
Registrati

Account demo:
Admin: admin@bugboard.com / admin123
Utente: user@bugboard.com / user123
Readonly: readonly@bugboard.com / readonly123

Figura 8: Dettaglio Bug: informazioni complete, commenti, storico modifiche e azioni disponibili

- **Scopo:** visualizzazione completa di un bug specifico e gestione delle azioni disponibili
- **Elementi:** header con titolo, stato e priorità; sezioni descrizione, commenti, allegati, storico; barra azioni laterale (modifica, assegna, risolvi, archivia, duplica)
- **Azioni visibili:** dipendono dal ruolo dell'utente (USER vede meno azioni rispetto ad Admin)

8.2.6 Mock-up 6 — Form Creazione/Modifica Bug [UC01]



BugBoard26
Gestione issue tracking per il tuo team

Email
nome@esempio.com

Password

[Accedi](#)

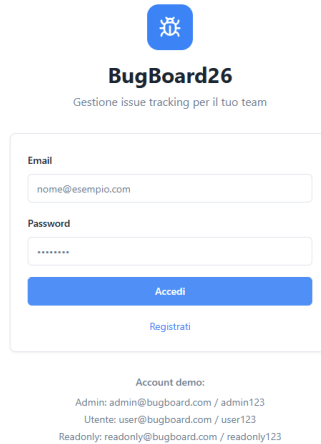
[Registrati](#)

Account demo:
Admin: admin@bugboard.com / admin123
Utente: user@bugboard.com / user123
Readonly: readonly@bugboard.com / readonly123

Figura 9: Form Creazione Bug (UC01): campi titolo, tipo, priorità, descrizione, label, allegati

- **Caso d'uso:** UC01 — Creazione nuovo bug
- **Elementi:** campo titolo, selettori tipo e priorità, area di testo descrizione, selezione label multipla, upload allegati immagine, selezione assegnatario opzionale, pulsante “Salva”
- **Validazione:** i campi obbligatori (titolo, tipo, priorità) sono evidenziati se lasciati vuoti; il formato e la dimensione degli allegati sono verificati prima dell'invio

8.2.7 Mock-up 7 — Assegnazione Bug [UC02]



BugBoard26
Gestione issue tracking per il tuo team

Email
nome@esempio.com

Password

Accedi

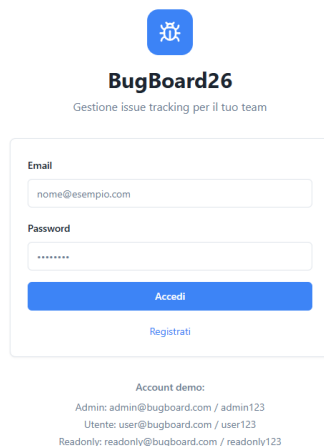
Registrati

Account demo:
Admin: admin@bugboard.com / admin123
Utente: user@bugboard.com / user123
Readonly: readonly@bugboard.com / readonly123

Figura 10: Assegnazione Bug (UC02): selezione dell'utente a cui assegnare il bug

- **Caso d'uso:** UC02 — Assegnazione bug a un utente (solo Admin)
- **Elementi:** select per la scelta dell'utente assegnatario, pulsante conferma
- **Accesso:** questa schermata è accessibile solo agli utenti con ruolo Admin; un USER viene reindirizzato alla Dashboard

8.2.8 Mock-up 8 — Notifiche

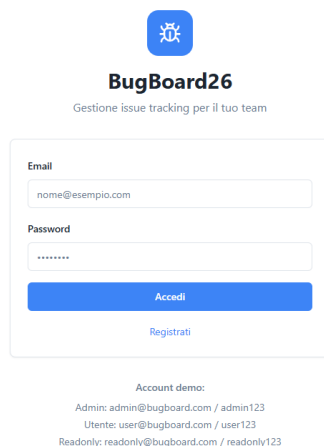


The mock-up shows a login interface for BugBoard26. At the top is a blue square icon with a white bug. Below it, the text "BugBoard26" is displayed in bold, followed by the tagline "Gestione issue tracking per il tuo team". The main form contains two input fields: "Email" with the placeholder "nome@esempio.com" and "Password" with masked characters "*****". A blue "Accedi" button is positioned below the password field, with a blue "Registrati" link underneath it. At the bottom, a section titled "Account demo:" lists three accounts: "Admin: admin@bugboard.com / admin123", "Utente: user@bugboard.com / user123", and "Readonly: readonly@bugboard.com / readonly123".

Figura 11: Centro Notifiche: lista degli aggiornamenti sui bug seguiti dall'utente

- **Scopo:** mostrare all'utente gli aggiornamenti recenti sui bug che lo riguardano (assegnazioni, cambi stato, nuovi commenti)
- **Elementi:** lista notifiche con timestamp, tipo evento e link al bug correlato; indicatore notifiche non lette

8.2.9 Mock-up 9 — Profilo Utente



This mock-up is identical to the one in Figure 11, showing the login form for BugBoard26. It includes the bug icon, the application name and tagline, the email and password input fields, the "Accedi" button, the "Registrati" link, and the "Account demo:" section with the three demo accounts.

Figura 12: Profilo Utente: gestione informazioni personali e cambio password

- **Scopo:** consentire all'utente di visualizzare e modificare le proprie informazioni e la password
- **Elementi:** nome, email, ruolo (in sola lettura), form cambio password con validazione

8.2.10 Mock-up 10 — Report Mensile [UC04]

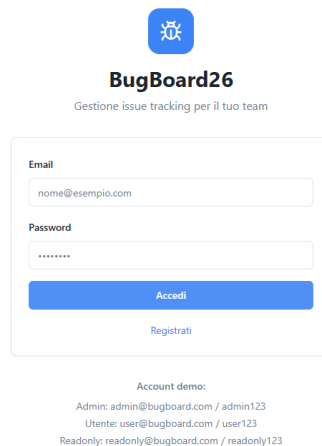
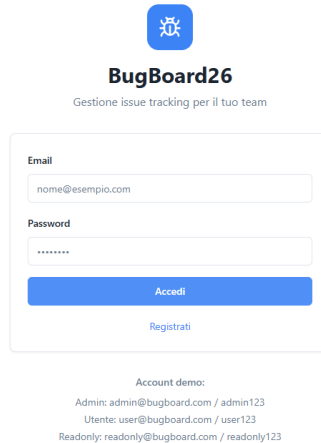


Figura 13: Report Mensile Admin (UC04): statistiche, grafici e pulsante di export Excel

- **Caso d'uso:** UC04 — Generazione report mensile (solo Admin)
- **Elementi:** selettore mese/anno, grafici (distribuzione stato, priorità, trend temporale), tabella riassuntiva metriche, pulsante “Esporta Excel”
- **Accesso:** solo Admin; gli altri utenti vengono reindirizzati alla Dashboard

8.2.11 Mock-up 11 — Gestione Utenti (Admin)

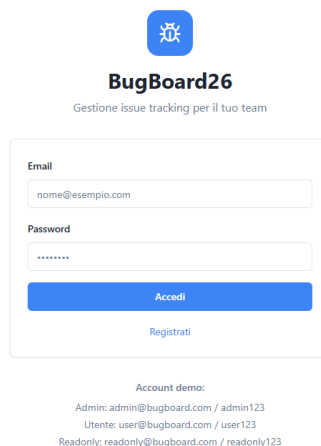


The mock-up shows the login page for BugBoard26. At the top is a blue square icon with a white bug. Below it is the text "BugBoard26" and "Gestione issue tracking per il tuo team". The login form has two input fields: "Email" with the placeholder "nome@esempio.com" and "Password" with masked characters. Below the fields are two buttons: a blue "Accedi" button and a blue "Registrati" link. At the bottom, there is a section titled "Account demo:" with three lines of text: "Admin: admin@bugboard.com / admin123", "Utente: user@bugboard.com / user123", and "Readonly: readonly@bugboard.com / readonly123".

Figura 14: Gestione Utenti Admin: lista utenti con ruoli, stato e azioni di modifica

- **Scopo:** permettere all'Admin di gestire gli account del sistema (visualizzazione ruoli, disabilitazione utenti)
- **Elementi:** tabella utenti con email, nome, ruolo e stato; azioni per modifica ruolo e disabilitazione account

8.2.12 Mock-up 12 — Archivio Bug



This mock-up is identical to the one in Figure 14, showing the login page for BugBoard26. It includes the bug icon, the title "BugBoard26", the subtitle "Gestione issue tracking per il tuo team", the login form with "Email" and "Password" fields, the "Accedi" button, the "Registrati" link, and the "Account demo:" section with demo credentials.

Figura 15: Archivio Bug: lista dei bug archiviati con possibilità di consultazione

- **Scopo:** visualizzare i bug archiviati per storico e consultazione senza influire sulla lista attiva
- **Elementi:** tabella bug in stato *Archived* con filtri e ricerca; solo Admin può archiviare/ripristinare bug

8.2.13 Mock-up 13 — Export Dati

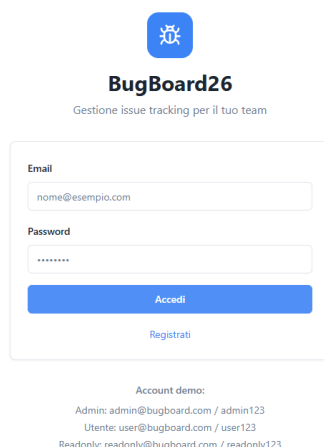


Figura 16: Export Dati: download dell'elenco bug in formato CSV o Excel

- **Scopo:** consentire l'esportazione dei dati per analisi esterne
- **Elementi:** pulsanti “Scarica CSV” e “Scarica Excel”; i file vengono generati dal backend e scaricati direttamente

8.3 Osservazioni emerse dalla prototipazione

La realizzazione del prototipo funzionante ha permesso di identificare alcune scelte migliorative rispetto all'analisi iniziale:

- **Chiarezza dei ruoli:** è stato necessario rendere esplicito nel UI quali azioni sono riservate all'Admin (assegnazione, report, archiviazione, gestione utenti), aggiungendo reindirizzamenti automatici per utenti non autorizzati
- **Filtri immediati:** la lista bug con filtri in tempo reale si è rivelata la funzionalità più usata nelle sessioni di test, confermando la scelta di renderla la pagina principale
- **Badge colorati:** l'uso di badge cromatici per stato, priorità e tipo ha migliorato significativamente la leggibilità della lista bug senza aprire il dettaglio
- **Navigazione mobile:** il menu hamburger con le stesse voci del desktop ha garantito piena parità funzionale su dispositivi mobili

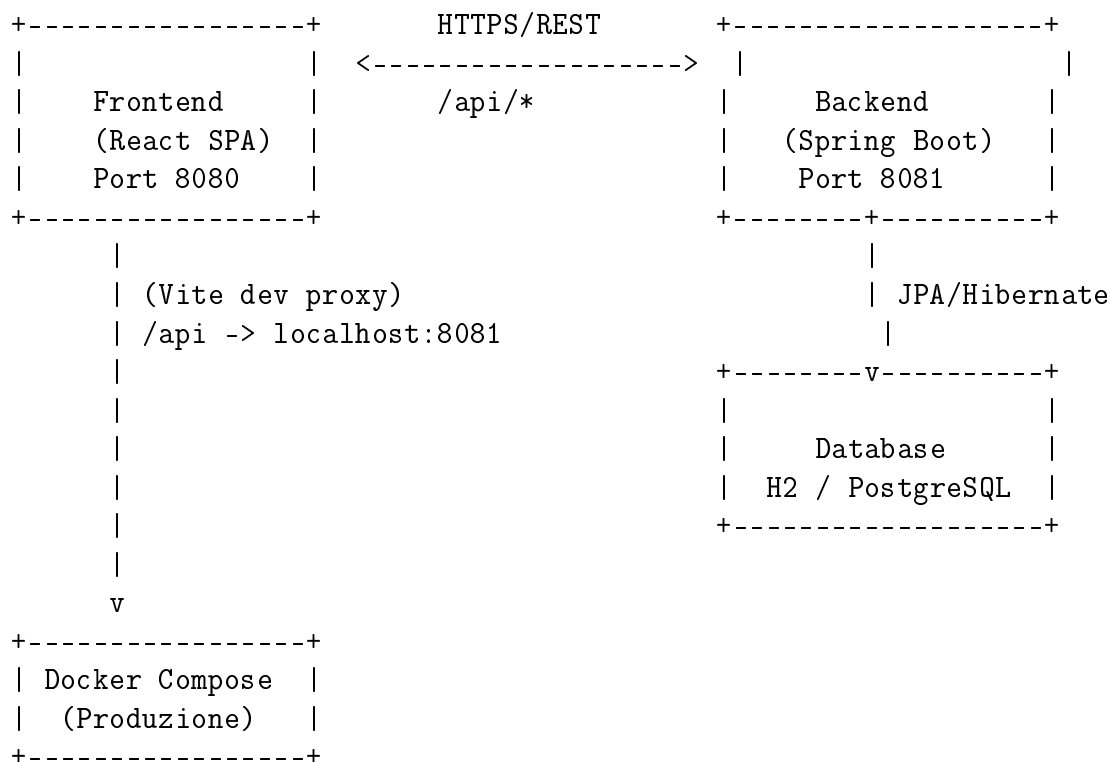
9 Design del Sistema

9.1 Descrizione dell'architettura

BugBoard26 adotta un'architettura **client-server a tre livelli** (Three-Tier Architecture), separando nettamente le responsabilità tra *presentazione*, *logica di business* e *persistenza dei dati*.

Livello	Descrizione
Presentazione (Frontend)	Single Page Application React che comunica con il backend tramite API REST.
Logica di Business (Backend)	Applicazione Spring Boot che espone endpoint RESTful, gestisce autenticazione JWT e implementa le regole di business.
Persistenza (Database)	Database relazionale (H2 in sviluppo, PostgreSQL in produzione) gestito tramite JPA/Hibernate.

9.1.1 Diagramma architetturale



9.2 Criteri di design adottati

Separazione delle responsabilità (*Separation of Concerns*) Ogni livello ha una responsabilità ben definita: il frontend si occupa esclusivamente della presentazione e dell'interazione utente, il backend della logica di business e dell'autorizzazione, il database della persistenza. Questa separazione favorisce la manutenibilità e la testabilità.

Stateless Authentication L'autenticazione è gestita tramite token JWT, eliminando la necessità di sessioni server-side. Ogni richiesta contiene il token necessario per l'identificazione, facilitando la scalabilità orizzontale.

API-First Design Il backend espone un'API RESTful documentata tramite OpenAPI/Swagger (SpringDoc). Questo approccio consente lo sviluppo indipendente di frontend e backend, facilita l'integrazione con altri client e permette il testing automatico delle API.

Convention over Configuration L'utilizzo di Spring Boot riduce drasticamente la configurazione boilerplate grazie alle *auto-configurations* e alle convenzioni del framework.

Containerizzazione L'intero stack è containerizzabile tramite Docker e orchestrabile con Docker Compose, garantendo riproducibilità dell'ambiente e facilità di deployment.

9.3 Scelte tecnologiche

9.3.1 Frontend

Tecnologia	Versione	Motivazione
React	18.x	Libreria UI component-based matura e con vasto ecosistema; facilita la creazione di SPA reattive.
TypeScript	5.x	Tipizzazione statica che riduce i bug a runtime e migliora la manutenibilità del codice.
Vite	6.x	Build tool moderno, significativamente più veloce di Webpack grazie a ESBuild e HMR nativo.
Tailwind CSS	3.x	Framework CSS utility-first che accelera lo sviluppo della UI e garantisce consistenza visiva.
Radix UI	–	Componenti accessibili e non stilizzati (<i>headless</i>), conformi a WAI-ARIA, personalizzabili con Tailwind.
React Query	5.x	Gestione dello stato server-side (caching, refetching, invalidation) che semplifica le chiamate API.
React Router	7.x	Routing dichiarativo per SPA con supporto a lazy loading e route protette.

9.3.2 Backend

Tecnologia	Versione	Motivazione
Java	23	Linguaggio robusto e type-safe, con record e pattern matching per DTO concisi.
Spring Boot	3.3.4	Framework enterprise standard per applicazioni Java; auto-configuration, starter dependencies, embedded server.
Spring Security	–	Gestione completa di autenticazione e autorizzazione; integrazione nativa con JWT.
Spring Data JPA	–	Astrazione ORM che elimina il codice boilerplate per l'accesso ai dati; supporto a Specification per query dinamiche.
H2 Database	–	Database embedded per lo sviluppo locale; nessuna installazione richiesta, avvio immediato.
PostgreSQL	16	Database relazionale per produzione; robusto, conforme ACID, supporto nativo a UUID.
JWT	0.12.5	Libreria per la creazione e validazione di token JWT; API fluente e sicura.
SpringDoc	2.6.0	Generazione automatica della documentazione OpenAPI 3.0 dalle annotazioni Spring.
Apache POI	5.3.0	Generazione di file Excel (.xlsx) per l'export dei dati bug.

9.3.3 Testing

Tecnologia	Motivazione
JUnit 5	Framework di testing standard per Java; supporto a test parametrizzati, display names, extensions.
Mockito 5	Libreria di mocking che consente l'isolamento delle dipendenze nei test di unità.
AssertJ	Asserzioni fluenti che migliorano la leggibilità dei test.

9.3.4 DevOps e Deployment

Tecnologia	Motivazione
Docker	Containerizzazione per ambienti riproducibili; Dockerfile multi-stage per build ottimizzati.
Docker Compose	Orchestrazione dei tre servizi (database, backend, frontend) con un solo comando.
Git / GitHub	Version control distribuito; collaborazione, code review, CI/CD integration.
Maven	Build automation per il backend; gestione dipendenze, life-cycle standardizzato.
npm	Package manager per il frontend; gestione dipendenze e script di build.

9.4 Comunicazione tra i livelli

La comunicazione tra frontend e backend avviene tramite API REST su HTTP/HTTPS:

- **Formato dati:** JSON (Content-Type: `application/json`)
- **Autenticazione:** Token JWT trasportato tramite cookie `HttpOnly` (`token`) oppure header `Authorization: Bearer <token>`
- **Proxy in sviluppo:** Vite dirige `/api/*` a `localhost:8081`, evitando problemi CORS durante lo sviluppo
- **Endpoint principali:**
 - `POST /api/auth/login` — Autenticazione
 - `GET/POST /api/bugs` — Listing e creazione bug
 - `PATCH /api/bugs/{id}` — Modifica bug
 - `POST /api/bugs/{id}/assign` — Assegnazione (admin)
 - `GET /api/admin/metrics` — Metriche (admin)
 - `GET /api/export/bugs.csv` — Export CSV

9.5 Sicurezza

- **Password:** hash BCrypt con salt automatico
- **CSRF:** disabilitato (architettura stateless con JWT)
- **CORS:** configurazione restrittiva con whitelist degli origin consentiti
- **Sessioni:** nessuna sessione server-side (`SessionCreationPolicy.STATELESS`)
- **Autorizzazione:** controllo a livello di servizio basato sul ruolo dell'utente (ADMIN, USER, READONLY)
- **Cookie JWT:** attributi `HttpOnly` e `SameSite=Lax` per mitigare XSS e CSRF

10 Design del Software

10.1 Schema per la persistenza dati

Il database relazionale è gestito tramite JPA/Hibernate con strategia `ddl-auto=update`. Di seguito lo schema entità-relazione con tutte le tabelle, colonne e vincoli.

10.1.1 Tabella users

Colonna	Tipo	Vincoli	Note
id	UUID	PK, AUTO	Identificativo univoco
email	VARCHAR	UNIQUE, NOT NULL	Indirizzo email
password_hash	CHAR(60)	NOT NULL	Hash BCrypt
name	VARCHAR	NOT NULL	Nome visualizzato
role	VARCHAR	NOT NULL	Enum: ADMIN, USER, READONLY
created_at	TIMESTAMP		Data di registrazione

10.1.2 Tabella bugs

Colonna	Tipo	Vincoli	Note
id	UUID	PK, AUTO	Identificativo univoco
title	VARCHAR	NOT NULL	Titolo del bug
description	TEXT	NOT NULL	Descrizione dettagliata
type	VARCHAR	NOT NULL	Enum: QUESTION, BUG, DOCUMENTATION, FEATURE
status	VARCHAR		Enum: TODO, IN_PROGRESS, RESOLVED, ARCHIVED
priority	VARCHAR		Enum: LOW, MEDIUM, HIGH, URGENT
archived	BOOLEAN	default false	Flag di archiviazione
created_at	TIMESTAMP		Data di creazione
deadline	TIMESTAMP	nullable	Scadenza opzionale
duplicate_of	UUID	FK → bugs.id	Bug duplicato (self-ref)
created_by	UUID	FK → users.id	Creatore
assignee_id	UUID	FK → users.id	Assegnatario

10.1.3 Tabella comments

Colonna	Tipo	Vincoli	Note
id	UUID	PK, AUTO	Identificativo univoco
text	TEXT	NOT NULL	Testo del commento
bug_id	UUID	FK → bugs.id, NOT NULL	Bug associato
author_id	UUID	FK → users.id, NOT NULL	Autore
created_at	TIMESTAMP		Data di creazione

10.1.4 Tabella history

Colonna	Tipo	Vincoli	Note
id	UUID	PK, AUTO	Identificativo univoco
bug_id	UUID	FK → bugs.id, NOT NULL	Bug associato
who_id	UUID	FK → users.id, NOT NULL	Utente esecutore
action	VARCHAR	NOT NULL	Enum: CREATE, UPDATE, ASSIGN, COMMENT, ARCHIVE, DUPLICATE, LABELS_SET, ATTACHMENT_UPLOADED
details	TEXT	nullable	Dettagli del cambiamento
created_at	TIMESTAMP		Timestamp dell'azione

10.1.5 Tabella labels

Colonna	Tipo	Vincoli	Note
id	UUID	PK, AUTO	Identificativo univoco
name	VARCHAR	UNIQUE, NOT NULL	Nome della label
created_at	TIMESTAMP		Data di creazione

10.1.6 Tabella di join bug_labels

Colonna	Tipo	Vincoli	Note
bug_id	UUID	FK → bugs.id	Relazione multi-a-molti Bug ↔ Label
label_id	UUID	FK → labels.id	

10.1.7 Diagramma E-R semplificato

```

users 1 ---< N bugs      (createdBy)
users 1 ---< N bugs      (assignee)
bugs  1 ---< N comments
bugs  1 ---< N history
users 1 ---< N comments  (author)
users 1 ---< N history   (who)
bugs  N >---< M labels  (bug_labels)
bugs  N ---> 1 bugs      (duplicateOf, self-referential)

```

10.2 Diagramma delle classi di design

Il diagramma delle classi di design è organizzato secondo il pattern architetturale **layered architecture** con i seguenti package:

`model` Entità JPA che mappano le tabelle del database: `User`, `Bug`, `Comment`, `History`, `Label` e le enumerazioni `Role`, `BugType`, `BugStatus`, `Priority`, `HistoryAction`.

10.3.3 Record Java per i DTO

I DTO sono implementati come `record` Java, che garantiscono immutabilità, auto-generazione di `equals/hashCode/toString`, e riduzione del codice boilerplate. Le annotazioni di validazione (`@NotBlank`, `@NotNull`, `@NotEmpty`) sono applicate direttamente sui componenti del record.

10.3.4 Specification Pattern per le query

La classe `BugSpecification` implementa il pattern Specification di Spring Data, componendo predicati JPA Criteria in modo dichiarativo. I filtri (tipo, stato, priorità, label, ricerca full-text su titolo e descrizione) possono essere combinati liberamente dal frontend.

10.3.5 Audit Trail tramite History

Ogni operazione significativa (creazione, modifica, assegnazione, commento, archiviazione) genera automaticamente una entry nella tabella `history`, con riferimento all'utente, all'azione e ai dettagli della modifica. Questo garantisce tracciabilità completa senza librerie esterne.

10.3.6 Autenticazione stateless con JWT

`JwtService` genera token firmati HMAC-SHA256 con claims personalizzati (ruolo, nome, email). `JwtAuthFilter` intercetta ogni richiesta, estrae il token da cookie o header, e popola il `SecurityContext` di Spring Security. Questo approccio elimina la necessità di sessioni server-side e facilita la scalabilità orizzontale.

10.3.7 Gestione delle autorizzazioni

Il controllo degli accessi è implementato a due livelli:

- **Livello HTTP:** Spring Security filtra le richieste in base all'autenticazione (JWT valido).
- **Livello Service:** le regole di business (solo admin può assegnare; solo assegnatario o admin può cambiare stato) sono verificate nel servizio, lanciando `SecurityException` in caso di violazione.

10.4 Evidenza dell'uso di strumenti di versioning

Il codice sorgente del progetto è ospitato su un repository **GitHub** accessibile ai membri del team di sviluppo.

- **Repository:** <https://github.com/alessandroilprogrammatore/BugBoard26>
- **Release:** la release “Specifica dei requisiti” è stata pubblicata il 19 ottobre 2025
- **Branching strategy:** sviluppo su branch feature, merge su main tramite pull request
- **Commit:** messaggi descrittivi che seguono il formato convenzionale (`feat:`, `fix:`, `docs:`, `test:`)

11 Test Plan – Gestione Bug

11.1 Obiettivo

Il test plan descrive le attività di verifica previste per la funzionalità **Gestione Bug** del sistema BugBoard26. L'obiettivo è validare che le operazioni di creazione, assegnazione e modifica di un bug rispettino i requisiti funzionali e le regole di autorizzazione definite nella specifica.

11.2 Funzionalità sotto test

Le funzionalità coperte dal piano sono quelle esposte dal servizio **BugService**:

1. **Creazione di un bug** – metodo `create(BugCreateRequest, UUID)`
2. **Assegnazione di un bug** – metodo `assign(UUID, AssignRequest, UUID)`
3. **Modifica di un bug** – metodo `patch(UUID, BugPatchRequest, UUID, boolean)`

11.3 Approccio di testing

- **Livello:** test di unità (*unit test*).
- **Tecnica:** *black-box* (classi di equivalenza, valori limite) combinata con ispezione *white-box* (copertura dei rami decisionali interni).
- **Framework:** JUnit 5 + Mockito 5 (via `spring-boot-starter-test`).
- **Isolamento:** le dipendenze (repository JPA) sono sostituite da *mock objects* iniettati con `@Mock` / `@InjectMocks`, garantendo che i test verifichino esclusivamente la logica di business senza accesso al database.

11.4 Casi di test pianificati

11.4.1 `create(BugCreateRequest, UUID)`

ID	Scenario	Risultato atteso
TC-C1	Creazione con titolo, descrizione, tipo e priorità validi	Bug salvato; history CREATE registrata
TC-C2	UserId inesistente nel sistema	EntityNotFoundException
TC-C3	Request con lista di label (alcune esistenti, altre nuove)	Bug con label associate; label inesistenti create automaticamente

11.4.2 `assign(UUID, AssignRequest, UUID)`

ID	Scenario	Risultato atteso
TC-A1	Admin assegna bug a utente esistente	Assignee aggiornato; history ASSIGN registrata
TC-A2	Utente non-admin tenta l'assegnazione	SecurityException
TC-A3	BugId inesistente	EntityNotFoundException

11.4.3 patch(UUID, BugPatchRequest, UUID, boolean)

ID	Scenario	Risultato atteso
TC-P1	Admin aggiorna titolo e priorità	Campi aggiornati; history UPDATE registrata
TC-P2	Non-admin tenta di cambiare stato di bug altrui	<code>SecurityException</code>
TC-P3	Assegnatario modifica il proprio bug senza cambiare stato	Modifica applicata con successo

11.5 Criteri di ingresso e uscita

Ingresso: codice sorgente compilabile; dipendenze Maven risolte; ambiente Java 23 configurato.

Uscita: tutti i 9 test passano (BUILD SUCCESS) con 0 failures e 0 errors; copertura dei rami decisionali principali raggiunta.

11.6 Ambiente di esecuzione

- **JDK:** Amazon Corretto 23.0.2
- **Build tool:** Apache Maven con plugin Surefire 3.1.2
- **CI locale:** `mvn test` dalla directory `backend/`

12 Strategie di Testing

Questa sezione documenta le strategie adottate per la progettazione dei test di unità del servizio `BugService`, nucleo della logica di business dell'applicazione BugBoard26.

12.1 Metodi testati

Sono stati selezionati tre metodi non banali, ciascuno con almeno due parametri, come richiesto dalla specifica di progetto:

1. `create(BugCreateRequest request, UUID userId)` – 2 parametri
2. `assign(UUID bugId, AssignRequest request, UUID userId)` – 3 parametri
3. `patch(UUID id, BugPatchRequest request, UUID userId, boolean isAdmin)` – 4 parametri

12.2 Classi di equivalenza

Per ciascun metodo sono state individuate le seguenti classi di equivalenza.

12.2.1 create(BugCreateRequest, UUID)

Classe	Condizione	Validità	Test case
EC-C1	<code>userId</code> esistente; request con campi obbligatori validi, senza label	Valida	TC-C1
EC-C2	<code>userId</code> non presente nel database	Invalida	TC-C2
EC-C3	Request con lista di label non vuota (misto label esistenti e nuove)	Valida	TC-C3

12.2.2 assign(UUID, AssignRequest, UUID)

Classe	Condizione	Validità	Test case
EC-A1	Chiamante con ruolo <code>ADMIN</code> ; bug e assegnatario esistenti	Valida	TC-A1
EC-A2	Chiamante con ruolo <code>USER</code> (non admin)	Invalida	TC-A2
EC-A3	<code>bugId</code> non presente nel database	Invalida	TC-A3

12.2.3 patch(UUID, BugPatchRequest, UUID, boolean)

Classe	Condizione	Validità	Test case
EC-P1	<code>isAdmin=true</code> ; modifica di campi generici (titolo, priorità)	Valida	TC-P1
EC-P2	<code>isAdmin=false</code> ; cambio di <code>status</code> su bug assegnato ad altro utente	Invalida	TC-P2
EC-P3	<code>isAdmin=false</code> ; modifica effettuata dall'assegnatario stesso (senza cambio di stato)	Valida	TC-P3

12.3 Copertura strutturale

Oltre alle classi di equivalenza, i test sono stati progettati per coprire i rami decisionali principali (*branch coverage*) della logica di business:

- **create:**
 - Ramo: utente trovato vs. utente non trovato (`findById` → `Optional.empty`)
 - Ramo: lista label presente vs. assente
 - Ramo: label già esistente (`findByName` → `present`) vs. label nuova (`findByName` → `empty` → `save`)
- **assign:**
 - Ramo: bug trovato vs. bug non trovato
 - Ramo: chiamante admin vs. non-admin (controllo `role == ADMIN`)
- **patch:**

- Ramo: `isAdmin == true` → modifica consentita
- Ramo: `isAdmin == false && cambio status && utente ≠ assegnatario` → `SecurityException`
- Ramo: `isAdmin == false && utente == assegnatario` → modifica consentita
- Ramo: campi null nel `BugPatchRequest` → nessuna modifica per quel campo

12.4 Tecnica di isolamento: Mock Objects

Per isolare il *System Under Test* (SUT) dalle dipendenze esterne, tutti i repository JPA sono sostituiti da mock Mockito:

Dipendenza	Mock
BugRepository	@Mock
UserRepository	@Mock
CommentRepository	@Mock
LabelRepository	@Mock
HistoryRepository	@Mock

Questo approccio garantisce:

- **Determinismo:** i test non dipendono dallo stato di un database reale; ogni esecuzione produce lo stesso risultato.
- **Velocità:** nessun I/O di rete o disco; l'intera suite viene eseguita in meno di 2 secondi.
- **Granularità:** ogni test verifica un singolo comportamento della logica di servizio, facilitando la localizzazione dei difetti.

12.5 Riepilogo dei risultati

L'esecuzione della suite produce il seguente esito:

```
Tests run: 9, Failures: 0, Errors: 0, Skipped: 0
BUILD SUCCESS
```

Tutti i 9 casi di test passano con successo, confermando il corretto funzionamento della logica di business per le tre funzionalità testate.

13 Report di Qualità del Codice (Backend)

13.1 Strumenti di analisi utilizzati

L'analisi della qualità del codice è stata condotta utilizzando:

- **Analisi statica manuale** con revisione sistematica del codice
- **SonarLint** integrato nell'IDE per la rilevazione in tempo reale di code smell e bug pattern

- **Compilatore Java 23** con warning abilitati
- **JUnit 5 + Mockito** per la verifica della copertura funzionale

Nota: SonarQube Server non è stato configurato per questo progetto. L'analisi equivalente è stata condotta con gli strumenti sopra descritti.

13.2 Metriche generali

Metrica	Valore
File sorgente Java	38
Linee di codice (LOC)	~2000
Classi	18
Interfacce (Repository)	5
Enumerazioni	5
Record (DTO)	8
Package	7
Test di unità	9 (tutti passano)

13.3 Distribuzione del codice per package

Package	File	Contenuto
model	10	Entità JPA (Bug, User, History, Comment, Label) + 5 enumerazioni
service	5	BugService, AuthService, ExportService, AnalyticsService, BugSpecification
controller	5	AuthController, BugController, AdminController, ExportController, RestExceptionHandler
dto	8	Record Java per request/response
repository	5	Interfacce Spring Data JPA
config	2	SecurityConfig, DataSeeder
security	2	JwtService, JwtAuthFilter
root	1	Bugboard26Application (entry point)

13.4 Punti di forza

- **Struttura dei package chiara:** separazione netta tra model, service, controller, dto, repository, config e security, conforme al pattern layered architecture.
- **Uso di feature moderne di Java:** record per i DTO (immutabilità, riduzione del boilerplate), enumerazioni per i tipi con valori finiti.
- **Best practice Spring Boot:** auto-configuration, starter dependencies, gestione centralizzata delle eccezioni tramite `RestExceptionHandler`.
- **Separazione delle responsabilità:** controller “thin” che delegano immediatamente ai servizi; logica di business concentrata nei service.
- **Sicurezza:** autenticazione JWT stateless, password hash con BCrypt, configurazione CORS restrittiva.

13.5 Code smell e aree di miglioramento identificate

13.5.1 Severità alta

1. **Autenticazione hardcoded nel controller:** Il metodo `getCurrentUserId()` in `BugController` restituisce un UUID fisso anziché estrarlo dal token JWT. Analogamente, `isCurrentUserAdmin()` ritorna sempre `true`. Questo rappresenta un rischio di sicurezza che deve essere corretto prima del deployment.
2. **Endpoint `/me` incompleto:** L'endpoint `GET /api/auth/me` in `AuthController` non restituisce i dati dell'utente corrente in modo completo.

13.5.2 Severità media

1. **Metodo lungo:** `BugService.patch()` (~41 righe) gestisce molteplici responsabilità (validazione permessi, aggiornamento campi, gestione duplicati, storico). Potrebbe essere suddiviso in metodi più piccoli.
2. **Logica duplicata:** la ricerca dell'utente con lancio di `EntityNotFoundException` è ripetuta in più metodi di `BugService`. Estraibile in un metodo helper privato.
3. **Query inefficienti:** `AnalyticsService.getMetrics()` e `getMonthlyReport()` caricano tutti i bug in memoria anziché usare query aggregate a livello di database.
4. **Possibile `NullPointerException`:** in `BugService.patch()`, l'accesso a `bug.getAssignee().ge` non è protetto da null check nel caso in cui il bug non abbia un assegnatario.

13.5.3 Severità bassa

1. **Magic number:** il limite di 5 MB per gli allegati è espresso come `5 * 1024 * 1024` inline. Dovrebbe essere una costante con nome significativo.
2. **Codice commentato:** `BugService.uploadAttachment()` contiene righe commentate per il salvataggio su filesystem.
3. **Classe utility non finale:** `BugSpecification` è una classe di utility ma non è dichiarata `final` né ha un costruttore privato.

13.6 Riepilogo

Categoria	Conteggio	Severità
Bug critici (sicurezza)	2	Alta
Code smell strutturali	4	Media
Code smell minori	3	Bassa
Totale issue	9	

Valutazione complessiva: il backend presenta una buona organizzazione architetturale e un uso appropriato dei pattern di Spring Boot. Le issue critiche sono localizzate e note (autenticazione placeholder), pianificate per correzione prima del rilascio in produzione. Le issue di severità media e bassa riguardano refactoring incrementali che non impattano la correttezza funzionale.